

**Saisaran M 251801281-2025-AIDS** ▾**S2****Started on** Thursday, 27 August 2026, 10:46 AM**State** Finished**Completed on** Thursday, 27 August 2026, 10:51 AM**Time taken** 4 mins 43 secs**Marks** 1.00/1.00**Grade** **10.00** out of 10.00 (**100%**)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int firstZero(int arr[], int low, int high) {
4      if (low > high)
5          return -1;
6
7      int mid = low + (high - low) / 2;
8
9      if (arr[mid] == 0) {
10         if (mid == 0 || arr[mid - 1] == 1)
11             return mid;
12
13         return firstZero(arr, low, mid - 1);
14     }
15     return firstZero(arr, mid + 1, high);
16 }
17
18 int main() {
19     int m;
20     scanf("%d", &m);
21
22     int arr[m];
23
24     for (int i = 0; i < m; i++) {
25         scanf("%d", &arr[i]);
26     }
27
28     int index = firstZero(arr, 0, m - 1);
29
30     if (index == -1)
31         printf("0\n");
32     else
33         printf("%d\n", m - index);
34
35     return 0;
36 }
37
38
39

```

	Input	Expected	Got	
✓	5	2	2	✓
	1			
	1			
	1			
	0			
	0			

	Input	Expected	Got	
✓	10 1 1 1 1 1 1 1 1 1 1 1 1	0	0	✓
✓	8 0 0 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

**Saisaran M 251801281-2025-AIDS** ▾**S2****Started on** Thursday, 27 August 2026, 10:51 AM**State** Finished**Completed on** Thursday, 27 August 2026, 10:58 AM**Time taken** 6 mins 48 secs**Marks** 1.00/1.00**Grade** **10.00** out of 10.00 (**100%**)

Question 1 | Correct | Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int candidate = 0;
8      int count = 0;
9      int num;
10
11     for (int i = 0; i < n; i++) {
12         scanf("%d", &num);
13
14         if (count == 0) {
15             candidate = num;
16             count = 1;
17         }
18         else if (num == candidate) {
19             count++;
20         }
21         else {
22             count--;
23         }
24     }
25
26     printf("%d\n", candidate);
27
28     return 0;
29 }
30

```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

[Saisaran M 251801281-2025-AIDS](#) ▾**S2****Started on** Thursday, 27 August 2026, 10:58 AM**State** Finished**Completed on** Thursday, 27 August 2026, 11:05 AM**Time taken** 6 mins 41 secs**Marks** 1.00/1.00**Grade** 10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int findFloor(int arr[], int low, int high, int x) {
4      if (low > high)
5          return -1;
6
7      int mid = low + (high - low) / 2;
8
9      if (arr[mid] == x)
10         return arr[mid];
11
12     if (arr[mid] < x) {
13         int result = findFloor(arr, mid + 1, high, x);
14
15         if (result == -1)
16             return arr[mid];
17
18         return result;
19     }
20
21     return findFloor(arr, low, mid - 1, x);
22 }
23
24 int main() {
25     int n;
26     scanf("%d", &n);
27
28     int arr[n];
29
30     for (int i = 0; i < n; i++) {
31         scanf("%d", &arr[i]);
32     }
33
34     int x;
35     scanf("%d", &x);
36
37     int floor = findFloor(arr, 0, n - 1, x);
38
39     printf("%d\n", floor);
40
41     return 0;
42 }
43
```

	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓
✓	5 10 22 85 108 129 100	85	85	✓
✓	7 3 5 7 9 11 13 15 10	9	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

**Started on** Thursday, 27 August 2026, 11:05 AM**State** Finished**Completed on** Thursday, 27 August 2026, 11:08 AM**Time taken** 3 mins 22 secs**Marks** 1.00/1.00**Grade** 10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int findPair(int arr[], int left, int right, int x, int *a, int *b) {
4      // Base case: no elements left
5      if (left >= right)
6          return 0;
7
8      int sum = arr[left] + arr[right];
9
10     if (sum == x) {
11         *a = arr[left];
12         *b = arr[right];
13         return 1;
14     }
15
16     if (sum < x) {
17         // Need a larger sum
18         return findPair(arr, left + 1, right, x, a, b);
19     }
20
21     // Need a smaller sum
22     return findPair(arr, left, right - 1, x, a, b);
23 }
24
25 int main() {
26     int n, x;
27
28     scanf("%d", &n);
29
30     int arr[n];
31
32     for (int i = 0; i < n; i++) {
33         scanf("%d", &arr[i]);
34     }
35
36     scanf("%d", &x);
37
38     int a, b;
39
40     if (findPair(arr, 0, n - 1, x, &a, &b)) {
41         printf("%d\n", a);
42         printf("%d\n", b);
43     } else {
44         printf("No\n");
45     }
46
47     return 0;
48 }
49

```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Back to Course

[Saisaran M 251801281-2025-AIDS](#) ▾**S2****Started on** Thursday, 17 September 2026, 11:31 AM**State** Finished**Completed on** Thursday, 17 September 2026, 11:36 AM**Time taken** 4 mins 54 secs**Marks** 1.00/1.00**Grade** **10.00** out of 10.00 (**100%**)

Question 1 | Correct | Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5	12 34 67 78 98
67 34 12 98 78	

Answer:

```

1  #include <stdio.h>
2
3  void swap(int *a, int *b) {
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8
9  int partition(int arr[], int low, int high) {
10     int pivot = arr[high];
11     int i = low - 1;
12
13     for (int j = low; j < high; j++) {
14         if (arr[j] <= pivot) {
15             i++;
16             swap(&arr[i], &arr[j]);
17         }
18     }
19
20     swap(&arr[i + 1], &arr[high]);
21
22     return i + 1;
23 }
24
25 void quickSort(int arr[], int low, int high) {
26     if (low < high) {
27         int pi = partition(arr, low, high);
28
29         quickSort(arr, low, pi - 1);
30         quickSort(arr, pi + 1, high);
31     }
32 }
33
34 int main() {
35     int n;
36
37     scanf("%d", &n);
38
39     int arr[n];
40
41     for (int i = 0; i < n; i++) {
42         scanf("%d", &arr[i]);
43     }
44
45     quickSort(arr, 0, n - 1);
46
47     for (int i = 0; i < n; i++) {
48         if (i > 0)
49             printf(" ");
50         printf("%d", arr[i]);
51     }
52

```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)